

Alternative wording for GB315 and GB316

Document #: P2106R0
Date: 2020-02-13
Project: Programming Language C++
LWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

§ 1 Introduction

This paper provides alternative wording for NB comments [GB315](#) and [GB316](#), based on LEWG's direction in Prague.

§ 2 Wording

This wording is relative to [\[N4849\]](#).

Edit 25.4 [\[algorithm.syn\]](#) as indicated:

```
#include <initializer_list>

namespace std {

+ namespace ranges {
+
+   // [algorithms.results], algorithm result types
+   template<class I, class F>
+   struct in_fun_result;
+
+   template<class I1, class I2>
+   struct in_in_result;
+
+   template<class I, class O>
+   struct in_out_result;
+
+   template<class I1, class I2, class O>
+   struct in_in_out_result;
+
+   template<class I, class O1, class O2>
+   struct in_out_out_result;
+
+   template<class T>
+   struct min_max_result;
+
+   template<class I>
+   struct in_found_result;
+ }
```

```

+
[...]
```

```

namespace ranges {
    template<class I, class F>
+   using for_each_result = in_fun_result<I, F>;
-   struct for_each_result {
-       [[no_unique_address]] I in;
-       [[no_unique_address]] F fun;
-
-       template<class I2, class F2>
-           requires convertible_to<const I&, I2> && convertible_to<const F&, F2>
-           operator for_each_result<I2, F2>() const & {
-               return {in, fun};
-           }
-
-       template<class I2, class F2>
-           requires convertible_to<I, I2> && convertible_to<F, F2>
-           operator for_each_result<I2, F2>() && {
-               return {std::move(in), std::move(fun)};
-           }
-   };

    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
              indirectly_unary_invocable<projected<I, Proj>> Fun>
    constexpr for_each_result<I, Fun>
        for_each(I first, S last, Fun f, Proj proj = {});
    template<input_range R, class Proj = identity,
              indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
    constexpr for_each_result<safe_iterator_t<R>, Fun>
        for_each(R&& r, Fun f, Proj proj = {});
}

[...]
```

```

namespace ranges {
    template<class I1, class I2>
+   using mismatch_result = in_in_result<I1, I2>;
-   struct mismatch_result {
-       [[no_unique_address]] I1 in1;
-       [[no_unique_address]] I2 in2;
-
-       template<class II1, class II2>
-           requires convertible_to<const I1&, II1> && convertible_to<const I2&, II2>
-           operator mismatch_result<II1, II2>() const & {
-               return {in1, in2};
-           }
-
-       template<class II1, class II2>
-           requires convertible_to<I1, II1> && convertible_to<I2, II2>
-           operator mismatch_result<II1, II2>() && {
-               return {std::move(in1), std::move(in2)};
-           }
-   };
};

```

```

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 =
        identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr mismatch_result<I1, I2>
    mismatch(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
            Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 =
        identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1,
        Proj2>
constexpr mismatch_result<safe_iterator_t<R1>, safe_iterator_t<R2>>
    mismatch(R1&& r1, R2&& r2, Pred pred = {},
            Proj1 proj1 = {}, Proj2 proj2 = {});
}

```

[...]

```

namespace ranges {
    template<class I, class O>
+   using copy_result = in_out_result<I, O>;
-   struct copy_result {
-       [[no_unique_address]] I in;
-       [[no_unique_address]] O out;
-   };
-   template<class I2, class O2>
-       requires convertible_to<const I&, I2> && convertible_to<const O&, O2>
-       operator copy_result<I2, O2>() const & {
-           return {in, out};
-       };
-   template<class I2, class O2>
-       requires convertible_to<I, I2> && convertible_to<O, O2>
-       operator copy_result<I2, O2>() && {
-           return {std::move(in), std::move(out)};
-       };
-   };

```

```

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O>
requires indirectly_copyable<I, O>
constexpr copy_result<I, O>
    copy(I first, S last, O result);
template<input_range R, weakly_incrementable O>
requires indirectly_copyable<iterator_t<R>, O>
constexpr copy_result<safe_iterator_t<R>, O>
    copy(R&& r, O result);
}

```

[...]

```

namespace ranges {
    template<class I, class O>
-   using copy_n_result = copy_result<I, O>;
+   using copy_n_result = in_out_result<I, O>;

```

```

template<input_iterator I, weakly_incrementable O>
    requires indirectly_copyable<I, O>
    constexpr copy_n_result<I, O>
        copy_n(I first, iter_difference_t<I> n, O result);
}

```

[...]

```

namespace ranges {
    template<class I, class O>

```

```

-    using copy_if_result = copy_result<I, O>;

```

```

+    using copy_if_result = in_out_result<I, O>;

```

```

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O, class Proj
    = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
    requires indirectly_copyable<I, O>
    constexpr copy_if_result<I, O>
        copy_if(I first, S last, O result, Pred pred, Proj proj = {});
template<input_range R, weakly_incrementable O, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires indirectly_copyable<iterator_t<R>, O>
    constexpr copy_if_result<safe_iterator_t<R>, O>
        copy_if(R&& r, O result, Pred pred, Proj proj = {});
}

```

[...]

```

namespace ranges {
    template<class I1, class I2>

```

```

-    using copy_backward_result = copy_result<I1, I2>;

```

```

+    using copy_backward_result = in_out_result<I1, I2>;

```

```

template<bidirectional_iterator I1, sentinel_for<I1> S1, bidirectional_iterator
    I2>
    requires indirectly_copyable<I1, I2>
    constexpr copy_backward_result<I1, I2>
        copy_backward(I1 first, S1 last, I2 result);
template<bidirectional_range R, bidirectional_iterator I>
    requires indirectly_copyable<iterator_t<R>, I>
    constexpr copy_backward_result<safe_iterator_t<R>, I>
        copy_backward(R&& r, I result);
}

```

[...]

```

namespace ranges {
    template<class I, class O>

```

```

-    using move_result = copy_result<I, O>;

```

```

+    using move_result = in_out_result<I, O>;

```

```

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O>
    requires indirectly_movable<I, O>
    constexpr move_result<I, O>
        move(I first, S last, O result);

```

```

template<input_range R, weakly_incrementable O>
    requires indirectly_movable<iterator_t<R>, O>
    constexpr move_result<safe_iterator_t<R>, O>
        move(R&& r, O result);
}

```

[...]

```

namespace ranges {
    template<class I1, class I2>
-   using move_backward_result = copy_result<I1, I2>;
+   using move_backward_result = in_out_result<I1, I2>;
}

```

```

template<bidirectional_iterator I1, sentinel_for<I1> S1, bidirectional_iterator
    I2>
    requires indirectly_movable<I1, I2>
    constexpr move_backward_result<I1, I2>
        move_backward(I1 first, S1 last, I2 result);
template<bidirectional_range R, bidirectional_iterator I>
    requires indirectly_movable<iterator_t<R>, I>
    constexpr move_backward_result<safe_iterator_t<R>, I>
        move_backward(R&& r, I result);
}

```

[...]

```

namespace ranges {
    template<class I1, class I2>
-   using swap_ranges_result = mismatch_result<I1, I2>;
+   using swap_ranges_result = in_in_result<I1, I2>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2>
        requires indirectly_swappable<I1, I2>
        constexpr swap_ranges_result<I1, I2>
            swap_ranges(I1 first1, S1 last1, I2 first2, S2 last2);
    template<input_range R1, input_range R2>
        requires indirectly_swappable<iterator_t<R1>, iterator_t<R2>>
        constexpr swap_ranges_result<safe_iterator_t<R1>, safe_iterator_t<R2>>
            swap_ranges(R1&& r1, R2&& r2);
}

```

[...]

```

namespace ranges {
    template<class I, class O>
-   using unary_transform_result = copy_result<I, O>;
+   using unary_transform_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, weakly_incrementable O,
        copy_constructible F, class Proj = identity>
        requires writable<O, indirect_result_t<F&, projected<I, Proj>>>
        constexpr unary_transform_result<I, O>
            transform(I first1, S last1, O result, F op, Proj proj = {});
    template<input_range R, weakly_incrementable O, copy_constructible F,
        class Proj = identity>

```

```

requires writable<0, indirect_result_t<F&, projected<iterator_t<R>, Proj>>>
constexpr unary_transform_result<safe_iterator_t<R>, 0>
    transform(R&& r, 0 result, F op, Proj proj = {});

```

```

template<class I1, class I2, class O>
+ using binary_transform_result = in_in_out_result<I1, I2, O>;
- struct binary_transform_result {
-     [[no_unique_address]] I1 in1;
-     [[no_unique_address]] I2 in2;
-     [[no_unique_address]] O out;
-     template<class II1, class II2, class OO>
-     requires convertible_to<const I1&, II1> &&
-         convertible_to<const I2&, II2> && convertible_to<const O&, OO>
-     operator binary_transform_result<II1, II2, OO>() const & {
-         return {in1, in2, out};
-     }
-
-     template<class II1, class II2, class OO>
-     requires convertible_to<I1, II1> &&
-         convertible_to<I2, II2> && convertible_to<O, OO>
-     operator binary_transform_result<II1, II2, OO>() && {
-         return {std::move(in1), std::move(in2), std::move(out)};
-     }
- };

```

```

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2,
        weakly_incrementable O, copy_constructible F, class Proj1 = identity,
        class Proj2 = identity>
requires writable<0, indirect_result_t<F&, projected<I1, Proj1>,
        projected<I2, Proj2>>>
constexpr binary_transform_result<I1, I2, O>
    transform(I1 first1, S1 last1, I2 first2, S2 last2, O result,
        F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, weakly_incrementable O,
        copy_constructible F, class Proj1 = identity, class Proj2 = identity>
requires writable<0, indirect_result_t<F&, projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>>>
constexpr binary_transform_result<safe_iterator_t<R1>, safe_iterator_t<R2>, O>
    transform(R1&& r1, R2&& r2, O result,
        F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {});
}

```

[...]

```

namespace ranges {
    template<class I, class O>
-     using replace_copy_result = copy_result<I, O>;
+     using replace_copy_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, class T1, class T2,
            output_iterator<const T2&> O, class Proj = identity>
    requires indirectly_copyable<I, O> &&
        indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const
            T1*>
    constexpr replace_copy_result<I, O>

```

```

        replace_copy(I first, S last, O result, const T1& old_value, const T2&
new_value,
                    Proj proj = {});
template<input_range R, class T1, class T2, output_iterator<const T2& O,
        class Proj = identity>
requires indirectly_copyable<iterator_t<R>, O> &&
        indirect_binary_predicate<ranges::equal_to,
                                projected<iterator_t<R>, Proj>, const T1*>
constexpr replace_copy_result<safe_iterator_t<R>, O>
        replace_copy(R&& r, O result, const T1& old_value, const T2& new_value,
                    Proj proj = {});

template<class I, class O>
- using replace_copy_if_result = copy_result<I, O>;
+ using replace_copy_if_result = in_out_result<I, O>;

template<input_iterator I, sentinel_for<I> S, class T, output_iterator<const T&
O,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>>
Pred>
requires indirectly_copyable<I, O>
constexpr replace_copy_if_result<I, O>
        replace_copy_if(I first, S last, O result, Pred pred, const T& new_value,
                    Proj proj = {});
template<input_range R, class T, output_iterator<const T& O, class Proj =
identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, O>
constexpr replace_copy_if_result<safe_iterator_t<R>, O>
        replace_copy_if(R&& r, O result, Pred pred, const T& new_value,
                    Proj proj = {});
}

[...]
```

```

namespace ranges {
    template<class I, class O>
-    using remove_copy_result = copy_result<I, O>;
+    using remove_copy_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, weakly_incrementable O, class T,
            class Proj = identity>
    requires indirectly_copyable<I, O> &&
            indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const
T*>
    constexpr remove_copy_result<I, O>
        remove_copy(I first, S last, O result, const T& value, Proj proj = {});
    template<input_range R, weakly_incrementable O, class T, class Proj = identity>
    requires indirectly_copyable<iterator_t<R>, O> &&
            indirect_binary_predicate<ranges::equal_to,
                                projected<iterator_t<R>, Proj>, const T*>
    constexpr remove_copy_result<safe_iterator_t<R>, O>
        remove_copy(R&& r, O result, const T& value, Proj proj = {});

    template<class I, class O>
-    using remove_copy_if_result = copy_result<I, O>;

```

```

+ using remove_copy_if_result = in_out_result<I, 0>;

template<input_iterator I, sentinel_for<I> S, weakly_increamentable O,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>>
        Pred>
requires indirectly_copyable<I, 0>
constexpr remove_copy_if_result<I, 0>
    remove_copy_if(I first, S last, O result, Pred pred, Proj proj = {});
template<input_range R, weakly_increamentable O, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, 0>
constexpr remove_copy_if_result<safe_iterator_t<R>, 0>
    remove_copy_if(R&& r, O result, Pred pred, Proj proj = {});
}

[...]
```

```

namespace ranges {
    template<class I, class O>
- using unique_copy_result = copy_result<I, 0>;
+ using unique_copy_result = in_out_result<I, 0>;

    template<input_iterator I, sentinel_for<I> S, weakly_increamentable O, class Proj
        = identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
requires indirectly_copyable<I, 0> &&
    (forward_iterator<I> ||
     (input_iterator<O> && same_as<iter_value_t<I>, iter_value_t<O>>)) ||
    indirectly_copyable_storable<I, 0>)
constexpr unique_copy_result<I, 0>
    unique_copy(I first, S last, O result, C comp = {}, Proj proj = {});
template<input_range R, weakly_increamentable O, class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C =
        ranges::equal_to>
requires indirectly_copyable<iterator_t<R>, 0> &&
    (forward_iterator<iterator_t<R>> ||
     (input_iterator<O> && same_as<range_value_t<R>, iter_value_t<O>>)) ||
    indirectly_copyable_storable<iterator_t<R>, 0>)
constexpr unique_copy_result<safe_iterator_t<R>, 0>
    unique_copy(R&& r, O result, C comp = {}, Proj proj = {});
}

[...]
```

```

namespace ranges {
    template<class I, class O>
- using reverse_copy_result = copy_result<I, 0>;
+ using reverse_copy_result = in_out_result<I, 0>;

    template<bidirectional_iterator I, sentinel_for<I> S, weakly_increamentable O>
requires indirectly_copyable<I, 0>
constexpr reverse_copy_result<I, 0>
    reverse_copy(I first, S last, O result);
template<bidirectional_range R, weakly_increamentable O>
requires indirectly_copyable<iterator_t<R>, 0>
constexpr reverse_copy_result<safe_iterator_t<R>, 0>

```

```

        reverse_copy(R&& r, 0 result);
    }

    [...]

namespace ranges {
    template<class I, class O>
-   using rotate_copy_result = copy_result<I, O>;
+   using rotate_copy_result = in_out_result<I, O>;

    template<forward_iterator I, sentinel_for<I> S, weakly_increamentable O>
        requires indirectly_copyable<I, O>
        constexpr rotate_copy_result<I, O>
            rotate_copy(I first, I middle, S last, O result);
    template<forward_range R, weakly_increamentable O>
        requires indirectly_copyable<iterator_t<R>, O>
        constexpr rotate_copy_result<safe_iterator_t<R>, O>
            rotate_copy(R&& r, iterator_t<R> middle, O result);
}

    [...]

namespace ranges {
-   template<class I, class O> using partial_sort_copy_result = copy_result<I, O>;
+   template<class I, class O>
+   using partial_sort_copy_result = in_out_result<I, O>;

    template<input_iterator I1, sentinel_for<I1> S1,
        random_access_iterator I2, sentinel_for<I2> S2,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 =
            identity>
        requires indirectly_copyable<I1, I2> && sortable<I2, Comp, Proj2> &&
            indirect_strict_weak_order<Comp, projected<I1, Proj1>, projected<I2,
                Proj2>>
        constexpr partial_sort_copy_result<I1, I2>
            partial_sort_copy(I1 first, S1 last, I2 result_first, S2 result_last,
                Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, random_access_range R2, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
        requires indirectly_copyable<iterator_t<R1>, iterator_t<R2>> &&
            sortable<iterator_t<R2>, Comp, Proj2> &&
            indirect_strict_weak_order<Comp, projected<iterator_t<R1>, Proj1>,
                projected<iterator_t<R2>, Proj2>>
        constexpr partial_sort_copy_result<safe_iterator_t<R1>, safe_iterator_t<R2>>
            partial_sort_copy(R1&& r, R2&& result_r, Comp comp = {},
                Proj1 proj1 = {}, Proj2 proj2 = {});
}

    [...]

namespace ranges {
    template<class I, class O1, class O2>
+   using partition_copy_result = in_out_out_result<I, O1, O2>;
-   struct partition_copy_result {
-       [[no_unique_address]] I in;
-       [[no_unique_address]] O1 out1;

```

```

- [[no_unique_address]] O2 out2;
-
- template<class II, class O01, class O02>
-     requires convertible_to<const I&, II> &&
-     convertible_to<const O1&, O01> && convertible_to<const O2&, O02>
-     operator partition_copy_result<II, O01, O02>() const & {
-         return {in, out1, out2};
-     }
-
-
-     template<class II, class O01, class O02>
-     requires convertible_to<I, II> &&
-     convertible_to<O1, O01> && convertible_to<O2, O02>
-     operator partition_copy_result<II, O01, O02>() && {
-         return {std::move(in), std::move(out1), std::move(out2)};
-     }
- };

template<input_iterator I, sentinel_for<I> S,
        weakly_incrementable O1, weakly_incrementable O2,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>>
        Pred>
requires indirectly_copyable<I, O1> && indirectly_copyable<I, O2>
constexpr partition_copy_result<I, O1, O2>
    partition_copy(I first, S last, O1 out_true, O2 out_false, Pred pred,
                  Proj proj = {});
template<input_range R, weakly_incrementable O1, weakly_incrementable O2,
        class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, O1> &&
        indirectly_copyable<iterator_t<R>, O2>
constexpr partition_copy_result<safe_iterator_t<R>, O1, O2>
    partition_copy(R&& r, O1 out_true, O2 out_false, Pred pred, Proj proj = {});
}

[...]
```

```

namespace ranges {
    template<class I1, class I2, class O>
-     using merge_result = binary_transform_result<I1, I2, O>;
+     using merge_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
            sentinel_for<I2> S2,
            weakly_incrementable O, class Comp = ranges::less, class Proj1 =
            identity,
            class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr merge_result<I1, I2, O>
    merge(I1 first1, S1 last1, I2 first2, S2 last2, O result,
          Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, weakly_incrementable O, class Comp =
        ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr merge_result<safe_iterator_t<R1>, safe_iterator_t<R2>, O>
    merge(R1&& r1, R2&& r2, O result,

```

```

        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
}

[...]
```

```

namespace ranges {
    template<class I1, class I2, class O>
-   using set_union_result = binary_transform_result<I1, I2, O>;
+   using set_union_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
            sentinel_for<I2> S2,
            weakly_incremmentable O, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
    requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
    constexpr set_union_result<I1, I2, O>
        set_union(I1 first1, S1 last1, I2 first2, S2 last2, O result, Comp comp = {},
                  Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, input_range R2, weakly_incremmentable O,
            class Comp = ranges::less, class Proj1 = identity, class Proj2 =
            identity>
    requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
    constexpr set_union_result<safe_iterator_t<R1>, safe_iterator_t<R2>, O>
        set_union(R1&& r1, R2&& r2, O result, Comp comp = {},
                  Proj1 proj1 = {}, Proj2 proj2 = {});
}

[...]
```

```

namespace ranges {
    template<class I1, class I2, class O>
-   using set_intersection_result = binary_transform_result<I1, I2, O>;
+   using set_intersection_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
            sentinel_for<I2> S2,
            weakly_incremmentable O, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
    requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
    constexpr set_intersection_result<I1, I2, O>
        set_intersection(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                         Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, input_range R2, weakly_incremmentable O,
            class Comp = ranges::less, class Proj1 = identity, class Proj2 =
            identity>
    requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
    constexpr set_intersection_result<safe_iterator_t<R1>, safe_iterator_t<R2>, O>
        set_intersection(R1&& r1, R2&& r2, O result,
                         Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
}

[...]
```

```

namespace ranges {
    template<class I, class O>
-   using set_difference_result = copy_result<I, O>;

```

```

+ using set_difference_result = in_out_result<I, 0>;

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2,
        weakly_incrementable O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr set_difference_result<I1, 0>
    set_difference(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                  Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, weakly_incrementable O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 =
        identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr set_difference_result<safe_iterator_t<R1>, 0>
    set_difference(R1&& r1, R2&& r2, O result,
                  Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
}

[...]
```

```

namespace ranges {
    template<class I1, class I2, class O>
- using set_symmetric_difference_result = binary_transform_result<I1, I2, 0>;
+ using set_symmetric_difference_result = in_in_out_result<I1, I2, 0>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
            sentinel_for<I2> S2,
            weakly_incrementable O, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr set_symmetric_difference_result<I1, I2, 0>
    set_symmetric_difference(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                            Comp comp = {}, Proj1 proj1 = {},
                            Proj2 proj2 = {});
    template<input_range R1, input_range R2, weakly_incrementable O,
            class Comp = ranges::less, class Proj1 = identity, class Proj2 =
            identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr set_symmetric_difference_result<safe_iterator_t<R1>,
        safe_iterator_t<R2>, 0>
    set_symmetric_difference(R1&& r1, R2&& r2, O result, Comp comp = {},
                            Proj1 proj1 = {}, Proj2 proj2 = {});
}

[...]
```

```

namespace ranges {
    template<class T>
+ using minmax_result = min_max_result<T>;
- struct minmax_result {
-     [[no_unique_address]] T min;
-     [[no_unique_address]] T max;
-
-     template<class T2>
-     requires convertible_to<const T&, T2>

```

```

-     operator minmax_result<T2>() const & {
-         return {min, max};
-     }
-
-     template<class T2>
-         requires convertible_to<T, T2>
-         operator minmax_result<T2>() && {
-             return {std::move(min), std::move(max)};
-         }
-     };

template<class T, class Proj = identity,
        indirect_strict_weak_order<projected<const T*, Proj>> Comp =
        ranges::less>
constexpr minmax_result<const T>
    minmax(const T& a, const T& b, Comp comp = {}, Proj proj = {});
template<copyable T, class Proj = identity,
        indirect_strict_weak_order<projected<const T*, Proj>> Comp =
        ranges::less>
constexpr minmax_result<T>
    minmax(initializer_list<T> r, Comp comp = {}, Proj proj = {});
template<input_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp =
        ranges::less>
    requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
constexpr minmax_result<range_value_t<R>>
    minmax(R&& r, Comp comp = {}, Proj proj = {});
}

[...]
```

```

namespace ranges {
    template<class I>
-     using minmax_element_result = minmax_result<I>;
+     using minmax_element_result = min_max_result<I>;

    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
    constexpr minmax_element_result<I>
        minmax_element(I first, S last, Comp comp = {}, Proj proj = {});
    template<forward_range R, class Proj = identity,
            indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp =
            ranges::less>
    constexpr minmax_element_result<safe_iterator_t<R>>
        minmax_element(R&& r, Comp comp = {}, Proj proj = {});
}

[...]
```

```

namespace ranges {
    template<class I>
+     using next_permutation_result = in_found_result<I>;
-     struct next_permutation_result {
-         bool found;
-         I in;
-     };

```

```

template<bidirectional_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr next_permutation_result<I>
    next_permutation(I first, S last, Comp comp = {}, Proj proj = {});
template<bidirectional_range R, class Comp = ranges::less,
        class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr next_permutation_result<safe_iterator_t<R>>
    next_permutation(R&& r, Comp comp = {}, Proj proj = {});
}

template<class BidirectionalIterator>
constexpr bool prev_permutation(BidirectionalIterator first,
                                BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
constexpr bool prev_permutation(BidirectionalIterator first,
                                BidirectionalIterator last, Compare comp);

namespace ranges {
    template<class I>
-   using prev_permutation_result = next_permutation_result<I>;
+   using prev_permutation_result = in_found_result<I>;

    template<bidirectional_iterator I, sentinel_for<I> S, class Comp = ranges::less,
            class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr prev_permutation_result<I>
    prev_permutation(I first, S last, Comp comp = {}, Proj proj = {});
    template<bidirectional_range R, class Comp = ranges::less,
            class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr prev_permutation_result<safe_iterator_t<R>>
    prev_permutation(R&& r, Comp comp = {}, Proj proj = {});
}
}

```

Delete 25.2 [\[algorithms.requirements\]](#) p16:

- 16 ~~The class templates `binary_transform_result`, `for_each_result`, `minmax_result`, `mismatch_result`, `next_permutation_result`, `copy_result`, and `partition_copy_result` have the template parameters, data members, and special members specified below. They have no base classes or members other than those specified.~~

Add a new subclause after 25.4 [\[algorithm.syn\]](#), or alternatively under 25.2 [\[algorithms.requirements\]](#), as follows:

§ **?.? Algorithm result types [\[algorithms.results\]](#)**

- 1 Each of the class templates specified in this subclause has the template parameters, data members, and special members specified below, and has no base classes or members other than those specified.

```

template<class I, class F>
struct in_fun_result {
    [[no_unique_address]] I in;
    [[no_unique_address]] F fun;

    template<class I2, class F2>
    requires convertible_to<const I&, I2> && convertible_to<const F&, F2>
    constexpr operator in_fun_result<I2, F2>() const & {
        return {in, fun};
    }

    template<class I2, class F2>
    requires convertible_to<I, I2> && convertible_to<F, F2>
    constexpr operator in_fun_result<I2, F2>() && {
        return {std::move(in), std::move(fun)};
    }
};

template<class I1, class I2>
struct in_in_result {
    [[no_unique_address]] I1 in1;
    [[no_unique_address]] I2 in2;

    template<class II1, class II2>
    requires convertible_to<const I1&, II1> && convertible_to<const I2&, II2>
    constexpr operator in_in_result<II1, II2>() const & {
        return {in1, in2};
    }

    template<class II1, class II2>
    requires convertible_to<I1, II1> && convertible_to<I2, II2>
    constexpr operator in_in_result<II1, II2>() && {
        return {std::move(in1), std::move(in2)};
    }
};

template<class I, class O>
struct in_out_result {
    [[no_unique_address]] I in;
    [[no_unique_address]] O out;

    template<class I2, class O2>
    requires convertible_to<const I&, I2> && convertible_to<const O&, O2>
    constexpr operator in_out_result<I2, O2>() const & {
        return {in, out};
    }

    template<class I2, class O2>
    requires convertible_to<I, I2> && convertible_to<O, O2>
    constexpr operator in_out_result<I2, O2>() && {
        return {std::move(in), std::move(out)};
    }
};

template<class I1, class I2, class O>

```

```

struct in_in_out_result {
    [[no_unique_address]] I1 in1;
    [[no_unique_address]] I2 in2;
    [[no_unique_address]] O out;

    template<class II1, class II2, class OO>
        requires convertible_to<const I1&, II1> &&
            convertible_to<const I2&, II2> && convertible_to<const O&, OO>
        constexpr operator in_in_out_result<II1, II2, OO>() const & {
            return {in1, in2, out};
        }

    template<class II1, class II2, class OO>
        requires convertible_to<I1, II1> &&
            convertible_to<I2, II2> && convertible_to<O, OO>
        constexpr operator in_in_out_result<II1, II2, OO>() && {
            return {std::move(in1), std::move(in2), std::move(out)};
        }
};

template<class I, class O1, class O2>
struct in_out_out_result {
    [[no_unique_address]] I in;
    [[no_unique_address]] O1 out1;
    [[no_unique_address]] O2 out2;

    template<class II, class OO1, class OO2>
        requires convertible_to<const I&, II> &&
            convertible_to<const O1&, OO1> && convertible_to<const O2&, OO2>
        constexpr operator in_out_out_result<II, OO1, OO2>() const & {
            return {in, out1, out2};
        }

    template<class II, class OO1, class OO2>
        requires convertible_to<I, II> &&
            convertible_to<O1, OO1> && convertible_to<O2, OO2>
        constexpr operator in_out_out_result<II, OO1, OO2>() && {
            return {std::move(in), std::move(out1), std::move(out2)};
        }
};

template<class T>
struct min_max_result {
    [[no_unique_address]] T min;
    [[no_unique_address]] T max;

    template<class T2>
        requires convertible_to<const T&, T2>
        constexpr operator min_max_result<T2>() const & {
            return {min, max};
        }

    template<class T2>
        requires convertible_to<T, T2>
        constexpr operator min_max_result<T2>() && {

```

```

        return {std::move(min), std::move(max)};
    }
};

template<class I>
struct in_found_result {
    [[no_unique_address]] I in;
    bool found;

    template <class I2>
    requires convertible_to<const I&, I2>
    constexpr operator in_found_result<I2>() const & {
        return {in, found};
    }
    template <class I2>
    requires convertible_to<I, I2>
    constexpr operator in_found_result<I2>() && {
        return {std::move(in), found};
    }
};

```

Edit 25.7.12 [\[alg.permutation.generators\]](#) p4 as indicated:

4 *Returns:* Let B be **true** if a next permutation was found and otherwise **false**. Returns:

- (4.1) — B for the overloads in namespace std.
- (4.2) — { B, last } {last, B} for the overloads in namespace ranges.

Edit 25.7.12 [\[alg.permutation.generators\]](#) p9 as indicated:

9 *Returns:* Let B be **true** if a previous permutation was found and otherwise **false**. Returns:

- (9.1) — B for the overloads in namespace std.
- (9.2) — { B, last } {last, B} for the overloads in namespace ranges.

Edit 20.10.2 [\[memory.syn\]](#), header <memory> synopsis, as indicated:

```

namespace std {

    [...]

    namespace ranges {
        template<class I, class O>
        - using uninitialized_copy_result = copy_result<I, O>;
        + using uninitialized_copy_result = in_out_result<I, O>;
        template<input_iterator I, sentinel_for<I> S1,
                no_throw_forward_iterator O, no_throw_sentinel<O> S2>
                requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
                uninitialized_copy_result<I, O>
                uninitialized_copy(I ifirst, S1 ilast, O ofirst, S2 olast);
        template<input_range IR, no_throw_forward_range OR>
                requires constructible_from<range_value_t<OR>, range_reference_t<IR>>
                uninitialized_copy_result<safe_iterator_t<IR>, safe_iterator_t<OR>>
                uninitialized_copy(IR&& in_range, OR&& out_range);
    }
}

```

```

    template<class I, class O>
-   using uninitialized_copy_n_result = uninitialized_copy_result<I, O>;
+   using uninitialized_copy_n_result = in_out_result<I, O>;

    template<input_iterator I, no-throw-forward-iterator O, no-throw-sentinel<O> S>
        requires constructible_from<iter_value_t<O>, iter_reference_t<I>>>
        uninitialized_copy_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast);
}

namespace ranges {
    template<class I, class O>
-   using uninitialized_move_result = uninitialized_copy_result<I, O>;
+   using uninitialized_move_result = in_out_result<I, O>;
    template<input_iterator I, sentinel_for<I> S1,
        no-throw-forward-iterator O, no-throw-sentinel<O> S2>
        requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>>
        uninitialized_move_result<I, O>
        uninitialized_move(I ifirst, S1 ilast, O ofirst, S2 olast);
    template<input_range IR, no-throw-forward-range OR>
        requires constructible_from<range_value_t<OR>, range_rvalue_reference_t<IR>>>
        uninitialized_move_result<safe_iterator_t<IR>, safe_iterator_t<OR>>>
        uninitialized_move(IR&& in_range, OR&& out_range);

    template<class I, class O>
-   using uninitialized_move_n_result = uninitialized_copy_result<I, O>;
+   using uninitialized_move_n_result = in_out_result<I, O>;
    template<input_iterator I,
        no-throw-forward-iterator O, no-throw-sentinel<O> S>
        requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>>
        uninitialized_move_n_result<I, O>
        uninitialized_move_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast);
}

[...]
```

If [P1243R4](#) is applied, edit [algorithm.syn] as indicated, immediately before the declaration of `ranges::for_each_n`:

```

namespace std {

[...]
```

```

    namespace ranges {
+   template<class I, class F>
+   using for_each_n_result = in_fun_result<I, F>;

    template<input_iterator I, class Proj = identity,
        indirectly_unary_invocable<projected<I, Proj>> Fun>
        constexpr for_each_n_result<I, Fun>
        for_each_n(I first, iter_difference_t<I> n, Fun f, Proj proj = {});
    }

```

```
[...]  
}
```

§ 3 References

[N4849] Richard Smith. 2020. Working Draft, Standard for Programming Language C++. <https://wg21.link/n4849>